

Capstone Project Review Questions - Custom Answers

Prepared responses (sample answers ignored)

1. What problem does your project solve?

Answer: My project solves day-to-day project tracking problems for teams. Instead of scattered notes and status updates, it gives one place to manage users, projects, tasks, status, ownership, and progress with role-based access controls.

2. What were the main objectives of your project?

Answer: My goals were to build a complete full-stack app, secure it with authentication and roles, support real CRUD workflows, visualize progress on a dashboard, and deploy it in containerized environments. I also wanted strong testing and operational scripts for repeatable setup.

3. Explain your application's architecture.

Answer: It's a three-tier architecture: React + TypeScript on the frontend, Node.js/Express + TypeScript on the backend, and MongoDB for persistence. The frontend calls REST endpoints, the backend enforces auth/validation/business rules, and MongoDB stores users, projects, tasks, and audit logs.

4. Walk us through a request from React to MongoDB and back.

Answer: A user clicks in React, which triggers an API call through Axios. The Express route receives the request, runs middleware like auth and validation, then executes model logic against MongoDB using Mongoose. The API returns JSON, and React updates state and re-renders the UI.

5. What is the role of React in your application?

Answer: React handles the user experience: routing, forms, table views, filtering, editing, and conditional rendering by role. I used component structure and hooks to keep state and network behavior manageable and predictable.

6. Why did you use Node.js and Express?

Answer: Node.js let me keep a consistent JavaScript/TypeScript stack end to end. Express gave me a clean way to organize REST routes, middleware, error handling, and cross-cutting concerns like authentication and auditing.

7. What is MongoDB's role?

Answer: MongoDB is the source of truth for persistent data. It stores users, projects, tasks, nested task details like notes/subtasks, and audit events. Mongoose schemas enforce model structure and validation rules.

8. What happens when a user logs in?

Answer: The client sends email/password to the login endpoint. The backend validates credentials, generates a JWT, and returns token + user profile. The frontend stores session info and includes the token on protected API requests.

9. What is JWT and why did you use it?

Answer: JWT is a signed token representing an authenticated session. I used it because it's simple for stateless API auth, works well for role-based checks, and keeps protected endpoints secure without server-side session storage.

10. What happens if an unauthenticated user calls a protected API?

Answer: Auth middleware rejects the request before route logic runs. The API returns 401 for missing/invalid token, and the frontend redirects or shows an authorization error.

11. What middleware does your application use?

Answer: I use JSON parsing, CORS, auth middleware, role authorization, request validation,

centralized error handling, and audit middleware that logs mutating actions and metadata for traceability.

12. Explain one CRUD operation end to end.

Answer: For task creation: the user fills the form in React and submits. Frontend sends POST /api/tasks with token. Backend validates fields and permissions, writes the task in MongoDB, returns the created document, and the UI refreshes the task list.

13. What HTTP methods did you use?

Answer: GET for reads, POST for create actions, PUT/PATCH for updates, and DELETE for removals. I used method semantics consistently across resources like users, projects, tasks, and audit endpoints.

14. What HTTP status codes should your application return?

Answer: I return 200 for success, 201 for created, 204 for successful delete with no body, 400 for validation problems, 401 for auth failures, 403 for forbidden role access, 404 for missing resources, and 500 for unexpected server errors.

15. How did you validate user input?

Answer: Validation is done server-side using route-level validators and schema-level checks. Frontend also validates for UX, but backend validation is the enforcement layer so bad requests can't bypass controls.

16. How do you handle errors?

Answer: Routes throw or return structured errors, and centralized error middleware standardizes responses. On the frontend I catch API failures and show useful messages instead of silent failures.

17. How did you test your APIs?

Answer: I used integration tests and targeted middleware/model unit tests with Vitest. I also validated behavior with manual endpoint checks and app-level flows to confirm auth, CRUD, and role restrictions work end-to-end.

18. How can you prove that data is actually being stored in MongoDB?

Answer: During demo I can create/update a record in the UI, then query Mongo in the container or via tooling and show the document is present with matching fields and timestamps.

19. How did Git help your team?

Answer: Git gave us safe branching for features, reviewable commits, and controlled merges into main. It also made conflict resolution and rollback practical when multiple streams of work overlapped.

20. What does npm do in your project?

Answer: npm manages dependencies and scripts for install, build, test, seed, and run tasks. It's also how both frontend and backend test/build pipelines are executed consistently.

21. What was one significant technical challenge?

Answer: One major challenge was merging long-lived feature work into an updated main branch without regressions. I resolved route/UI conflicts, fixed post-merge test failures, and verified everything with backend/frontend test runs and dev reseeding.

22. How did your team troubleshoot problems?

Answer: We debugged by layer: verify frontend request payload, inspect network response, check backend logs/middleware behavior, then validate database state. That narrowed root causes quickly and avoided guessing.

23. What did you know before the project compared with what you know now?

Answer: Before, I understood each technology in isolation. Now I can design and operate a complete system: auth flow, role enforcement, API design, persistence, testing, merge strategy, and containerized deployment.

24. What is the most important thing you learned from the project?

Answer: The biggest lesson was building for maintainability, not just functionality—clear route boundaries, predictable data contracts, meaningful tests, and operational scripts made the project reliable under change.

25. What part of the project best demonstrates that you applied what you learned?

Answer: The admin + audit feature set is the best example. It combines auth, role checks, frontend routing, backend middleware, database models, audit events, tests, and merge-safe integration.

26. Why should we believe this is more than just code that happens to work?

Answer: Because I can explain each decision, trace requests end to end, show test coverage and failure recovery, demonstrate seeded reproducible environments, and prove role/audit controls with live scenarios.

27. If you had another week, what would you improve?

Answer: I'd add deeper automated coverage (especially cross-feature regressions), improve observability/structured logs, refine audit analytics UX, and harden deployment automation with more environment checks.

28. What would happen if MongoDB became unavailable?

Answer: Database-dependent routes would fail gracefully. The API should return controlled errors, avoid leaking internals, and allow health checks/alerts to surface the outage so the service can recover safely.

29. Why shouldn't React communicate directly with MongoDB?

Answer: Because the backend is the security and business-rules boundary. It enforces authentication, authorization, validation, auditing, and data-shape control. Direct browser-to-DB access would bypass all of that.

30. Can you explain your entire project in 30 seconds?

Answer: I built a full-stack task tracker using React, Express, and MongoDB with JWT auth, role-based permissions, admin management, dashboard insights, and a full audit pipeline. It runs in containerized dev/prod setups, is test-backed, and demonstrates real-world end-to-end engineering.

31. What did we build?

Answer: We built a role-aware project and task tracking platform with admin controls, audit visibility, and operational tooling for local/dev/prod workflows.

32. How does it work?

Answer: Users authenticate, interact through React pages, and all actions go through secured Express APIs to MongoDB. The app enforces permissions and logs important events for accountability.

33. What did I personally contribute?

Answer: I contributed heavily in backend route/middleware work, conflict resolution across branches, integrating audit tracking into frontend and backend, and validating the final state with rebuild/reseed/testing.

34. What problem did we encounter and solve?

Answer: We hit merge conflicts and post-merge regressions between admin and audit feature streams. I resolved conflicting code paths, fixed failing frontend tests, and verified clean mergeability into main.

35. What can I do now that I could not do before this training/project?

Answer: I can take a feature from idea to production-style implementation: design it, build full-stack behavior, secure it, test it, merge it safely, run it in containers, and explain every layer with confidence.